# 챕터 5. 주소만 내주는 업로드. Presigned URL + S3 + Lambda

**이번 챕터가 끝나면**

- 왜 사진을 서버가 직접 받아 쥐지 않고 **주소만** 내주는지 이해합니다 (Presigned URL의 의도)
- Spring이 발급한 임시 주소로 **클라이언트가 S3에 곧바로 PUT 업로드**하는 흐름을 손으로 만들어 봅니다
- S3 업로드가 끝나면 **Lambda가 자동으로 리사이즈**해서 `resized/` 폴더에 결과를 떨어뜨리는 파이프라인을 구성합니다
- 서버는 원본 업로드를 한 번도 받지 않고도, **key 규칙만 믿고** DB에 메타데이터를 심습니다

**이번 챕터는 "호텔 프런트가 짐을 받지 않고, 방 키만 내주는 이야기"입니다**

챕터 4에서 사진을 글자로 바꿔 JSON에 태워 보내는 방식을 손으로 해 보고, 10MB 앞에서 그 방식이 무너지는 지점을 숫자로 확인했습니다. 팀장이 복도에서 남긴 한마디는 "서버가 직접 받지 말고, 주소만 내주라"였습니다. 그 한마디가 이 챕터의 출발점입니다. 서버가 10MB짜리 바이트 덩어리를 자기 힘 위에 한 번도 올리지 않고, **업로드 주소 한 줄만** 발급해서 내주는 구조를 만들어 보겠습니다. 거기에 **자동 리사이즈**를 붙여, 사진이 S3에 닿는 순간 Lambda가 알아서 썸네일을 깎아 두는 작은 자동화까지 얹습니다.

**준비하기. 실습 시작 전 한 번만 설정**

### 1. 소스 코드 준비

챕터 5는 Spring Boot 하나로 시작합니다. 단, 이번에는 **AWS 쪽 세팅**이 실습의 절반 이상을 차지합니다. S3 버킷, Lambda 함수, IAM Access Key 세 가지가 서로 맞물려야 전체 흐름이 돕니다.

| 레포 | 용도 | 주소 |
| --- | --- | --- |
| **spring-presign-url** | Spring Boot + S3 Presigner + H2 인메모리 DB | `github.com/metacoding-11-spring-reference/spring-presign-url-start` |

터미널에서 클론합니다.

터미널 실습 레포 클론

```
git clone https://github.com/metacoding-11-spring-reference/spring-presign-url-start.git
cd spring-presign-url-start
```

파일 구조는 이렇습니다.

spring-presign-url 디렉토리

```
spring-presign-url/
├── build.gradle                                  # [실습] AWS SDK v2 의존성 추가
├── .env                                          # [실습] ACCESS_KEY·SECRET_KEY·BUCKET
├── src/main/
│   ├── java/com/metacoding/spring_presign_url/
│   │   ├── _core/config/
│   │   │   ├── AwsS3Config.java                  # [실습] S3Client·S3Presigner 빈 등록
│   │   │   └── CorsConfig.java                   # [참고] 브라우저 호출용 CORS
│   │   └── image/
│   │       ├── ImageController.java              # [실습] /presigned · /complete · /list · /{id}
│   │       ├── ImageService.java                 # [실습] Presign 발급 + 완료 저장 + 조회
│   │       ├── ImageEntity.java                  # [실습] image_tb (originalUrl · resizedUrl)
│   │       ├── ImageRequest.java                 # [실습] PresignRequest · CompleteRequest
│   │       ├── ImageResponse.java                # [실습] PresignedUrlResponse · Detail · Items
│   │       └── ImageRepository.java              # [참고] JpaRepository
```

```
│   └── resources/
│       └── application.properties              # [실습] cloud.aws.* 매핑
└── lambda/
    └── image_resize_handler.py                 # [실습] Lambda에 Deploy하는 파이썬 코드
```

> **이 레포는 챕터 4의 spring-base64와 별개입니다.** `uploads/` 폴더도, 로컬 파일시스템 저장도 이제 쓰지 않습니다. 사진의 진짜 보관처는 S3 버킷이고, 서버는 그 사실을 **기록만** 합니다.

### 2. 실습 환경 구축

이번 챕터는 AWS 계정이 필수입니다. 가입·결제 수단 등록·리전 설정(서울, `ap-northeast-2`)까지 끝낸 상태를 전제로 합니다. 프리 티어 기준으로 이 실습은 과금이 거의 발생하지 않지만, Access Key가 외부에 노출되면 피해가 커집니다. `.env`는 반드시 `.gitignore`에 두고 커밋하지 마세요.

터미널 실습 환경 확인

```
java --version
./gradlew --version
aws --version    # 선택. AWS CLI가 있으면 S3 확인이 편합니다
```

### 3. 사용할 구성 요소

챕터 5에 새로 얹히는 재료는 다섯 가지입니다.

| 재료 | 역할 |
|---|---|
| `software.amazon.awssdk:s3` | AWS SDK v2. `S3Client`와 `S3Presigner`를 제공 |
| `S3Presigner` | Presigned URL을 **서명**해서 발급하는 전용 클라이언트 |
| AWS S3 | 원본 (`original/`)과 리사이즈 결과 (`resized/`) 두 폴더를 가진 버킷 |
| AWS Lambda (Python 3.11) | S3 이벤트를 받아 Pillow로 리사이즈한 뒤 `resized/`에 저장 |
| `Klayers-p311-Pillow` | Lambda 런타임에 없는 Pillow를 얹기 위한 공개 Layer |

**Presigned URL을 먼저 만드는 이유**

서버가 10MB짜리 사진을 자기 손으로 받아 쥐지 않는다는 선택은, 단순한 성능 최적화가 아니라 **책임의 분리**입니다. 파일을 받는 일은 S3가 훨씬 잘합니다. 그 위에서 "이미지가 바뀌면 썸네일도 바꿔라" 같은 부수 작업을 Lambda가 이벤트로 자동 처리합니다. 서버는 오직 **언제, 누구 것을, 어떤 규칙으로 부르는지** 만 책임집니다. 이 분리가 뒤 챕터(SSE·RabbitMQ)에서 "서버가 상태를 들고 있지 않아도 흐르는 파이프라인"의 토대가 됩니다.

### 4. 실습 순서

이번 챕터의 절은 이 순서로 진행합니다.

1. `5.1`. 서버가 10MB를 직접 받지 않겠다는 결심
2. `5.2`. 프런트에서 방 키만 내주기, 파이프라인 설계
3. `5.3`. AWS 기본 세팅 (S3 · IAM · Lambda 트리거)
4. `5.4`. Spring에 S3Presigner 얹기
5. `5.5`. Postman으로 Presigned URL 받고 S3에 직업로드
6. `5.6`. Lambda 리사이즈 함수 배포
7. `5.7`. Webhook 꿈꾸다 접기, 완료 저장 API 설계
8. `5.8`. Postman으로 전체 흐름 검증과 목록·상세 조회

1에서 10MB의 벽을 다시 한 번 마주합니다. 2에서 새 파이프라인을 그리고, 3~4에서 AWS와 Spring을 얹습니다. 5에서 첫 업로드를 성공시킨 뒤, 6에서 Lambda를 붙이고, 7에서 한 번 잘못된 설계로 돌아갔다가 본래 자리로 돌아옵니다. 8에서 전체가 한 바퀴 도는 걸 확인합니다.

## 5.1 서버가 10MB를 직접 받지 않겠다는 결심

챕터 4의 마지막, 10MB짜리 셀카가 힙 위에 올라앉아 서버를 밀어내던 오후였습니다.

오픈이는 새 프로젝트를 하나 더 만들었습니다. 이번에는 `spring-presign-url`이라는 이름이었습니다. IntelliJ가 Gradle을 당기는 동안, 옆자리 동료가 의자를 반 바퀴 돌렸습니다.

**동료** : "그래서 이제 사진을 서버로 안 보내요? 저는 어디로 보내요?"

**오픈이** : "S3로 바로 보내요. 우리 서버는 주소만 내줄 거예요."

**동료** : "주소요. 그냥 `https://s3.aws.com/버킷/cat.png` 이런 거요? 그럼 아무나 거기에 사진 올리는 거 아니에요?"

오픈이는 손이 잠시 멈췄습니다. 동료의 말이 맞았습니다. S3 버킷을 그냥 열어 두면 아무나 아무 이름으로 아무 파일을 올릴 수 있었습니다. 10MB는 해결되지만, 통제가 사라집니다.

서명된 주소여야 해.

그때 복도에서 커피를 타 오던 팀장이 멈췄습니다.

**팀장** : "호텔 프런트에 손님이 오면 직원이 직접 짐을 받아서 방까지 올려다 줘요? 아니면 방 키만 내주고 손님이 알아서 올라가게 해요?"

**오픈이** : "방 키만 주죠."

**팀장** : "그 방 키는 아무 방이나 열려요?"

**오픈이** : "아니요. 배정된 그 방만요."

**팀장** : "유효기간은요?"

**오픈이** : "체크아웃까지요."

팀장은 그 말만 남기고 자리로 돌아갔습니다.

배정된 한 방, 정해진 기간. 그게 Presigned URL이구나.

오픈이는 수첩을 폈습니다. 지금까지의 업로드는 프런트(서버)가 손님(클라이언트)의 짐(사진)을 직접 받아서 방(저장소)까지 들어다 주는 모양이었습니다. 바꿀 구조는 프런트가 **이 방에만, 15분 동안만** 짐을 직접 들일 수 있는 키를 한 장 내주고, 손님이 그 키로 직접 S3 객실 문을 여는 모양입니다. 서버는 짐의 무게를 한 번도 들지 않습니다.

그럼 주소는 어떻게 서명하지.

검색창에 "aws s3 presigned url java"를 쳤습니다. 첫 번째 결과는 AWS SDK v2의 `S3Presigner` 클래스였습니다. Access Key와 Secret Key로 서명된 **한 번 쓰고 버리는 URL**을 만들어 주는 전용 객체였습니다. URL 안에 서명(Signature)·만료시각(Expires)·객체 키(Key)·HTTP 메서드(PUT)가 전부 박혀 들어가 있었습니다.

서버는 이 주소를 발급만 한다. 실제 업로드는 클라이언트와 S3가 한다.

10MB가 서버 힙에 올라올 일이 사라졌습니다.

## 5.2 프런트에서 방 키만 내주기

Presigned URL은 **서명이 찍힌 한 번짜리 주소**입니다. 그 주소는 다음 네 가지를 모두 품고 있습니다.

- 어떤 버킷의
- 어떤 이름(`original/{uuid}.png`)으로
- 어떤 HTTP 메서드(PUT)로만
- 언제까지(15분) 쓸 수 있는지

이 네 가지가 URL 하나에 서명되어 있으니, 이 URL을 받은 클라이언트는 정확히 그 조건으로만 S3에 접근합니다. 버킷 이름을 바꿔 끼워 넣거나, 만료된 뒤 다시 써 보거나, 같은 URL로 다른 객체를 덮어쓰는 일은 전부 서명 검증 단계에서 거절됩니다.

[GEMINI PROMPT: 호텔 로비 장면. 왼쪽에 프런트 데스크, 데스크 위에 커다란 방 키카드 한 장이 놓여 있고 카드에는 "Room 302 / PUT / 15 min / original/uuid.png"이라는 글자가 적혀 있다. 프런트 직원(Spring 서버를 상징, 정장 차림) 이 손님(Client 캐릭터)에게 카드를 건네고, 손님은 그 카드를 들고 우측의 거대한 초록색 금고(S3 버킷)에 혼자 걸어가 짐(사진)을 직접 넣는다. 프런트 직원은 뒤돌아 서류(DB)에 무언가를 적기만 한다. 흰 배경, 인디고·오렌지 악센트, 부드러운 일러스트.]

그림 5-1. 서버는 방 키만 내주고, 손님(클라이언트)이 직접 객실(S3)에 짐을 넣습니다

업로드가 S3에 닿은 **순간**부터 자동화가 한 층 덧붙습니다. S3는 객체가 만들어졌다는 사실(ObjectCreated 이벤트)을 Lambda에 알리고, Lambda는 `original/` 폴더에 떨어진 파일만 골라 Pillow로 썸네일을 깎은 뒤 `resized/`에 다시 올립니다. 원본의 UUID를 그대로 써서 이름을 맞춥니다.

**그림 5-2. 한 장의 사진이 S3를 거쳐 DB에 한 행으로 맺히기까지**

| | |
|---|---|
| ① Client → Spring | `POST /presigned` 파일명·콘텐츠 타입 전달 |
| ② Spring → Client | 서명된 PUT URL + `key = original/{uuid}.ext` 응답 |
| ③ Client → S3 | Presigned URL로 원본을 직접 PUT (서버 우회) |

| | |
|---|---|
| ④ S3 → Lambda | ObjectCreated 이벤트가 Lambda 트리거로 전달 |
| ⑤ Lambda → S3 | 원본을 받아 Pillow로 800px 리사이즈 → `resized/{uuid}.jpg` 저장 |
| ⑥ Client → Spring | `POST /complete` 업로드 완료를 알림 + key 전달 |
| ⑦ Spring → DB | key에서 UUID 뽑아 originalUrl·resizedUrl 조합 후 `image_tb` 저장 |

사진의 바이너리는 서버 힙에 단 한 번도 올라오지 않습니다

오픈이는 수첩에 설계를 정리했습니다.

— 설계 정리 —

1. 발급 API (`POST /presigned`)
   - 요청: `fileName` · `contentType`
   - 응답: `presignedUrl` · `key` (`original/{uuid}.ext`)
2. 업로드
   - 클라이언트가 발급받은 URL로 직접 S3에 PUT
   - 헤더 `Content-Type`은 발급 요청의 `contentType`과 같아야 함
3. 자동 리사이즈
   - S3가 `original/` 경로의 ObjectCreated 이벤트를 Lambda로 전달
   - Lambda가 원본을 읽어 `resized/{uuid}.jpg`로 저장
4. 완료 저장 (`POST /complete`)
   - 요청: `fileName` · `key`
   - 서버는 key에서 UUID를 뽑아 `originalUrl` · `resizedUrl`을 조합해 DB에 심음
5. 조회 (`GET /list`, `GET /{id}`)
   - 목록은 가벼운 필드만, 상세는 메타 전부

이 설계를 코드로 옮기기 전에, AWS 쪽 세 덩어리(S3·IAM·Lambda 트리거)를 먼저 얹어야 합니다.

## 5.3 AWS 기본 세팅

AWS 콘솔 작업이 많습니다. 각 단계는 한 번만 하면 이후 챕터에서도 그대로 씁니다.

### 5.3.1 S3 버킷과 두 폴더

콘솔 검색창에 `S3`를 입력해 이동한 뒤, "버킷 만들기"로 새 버킷을 하나 만듭니다. 실습에서 공개 접근을 써야 하기 때문에 **모든 퍼블릭 차단 해제**를 체크합니다. 실서비스에서는 이 설정을 절대 그대로 두면 안 됩니다. 챕터 마지막 "이것만은 기억하자"에서 다시 짚습니다.

버킷이 만들어지면 버킷 안에 두 폴더를 생성합니다.

- `original/` (원본이 올라오는 자리)
- `resized/` (Lambda가 썸네일을 떨어뜨리는 자리)

권한 탭의 "버킷 정책"을 편집해 아래 JSON을 붙입니다. `metacoding-base64` 자리에는 본인 버킷 이름을 넣습니다.

실습 1 S3 버킷 정책. 실습용 전체 공개

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicAllAccess",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::metacoding-base64",
        "arn:aws:s3:::metacoding-base64/*"
      ]
    }
  ]
}
```

`"Principal": "*"`는 "누구나"라는 뜻입니다. 실서비스였다면 Cognito·Federated Identity로 사용자를 특정하거나, 업로드는 Presigned URL 서명에만 의존하고 버킷은 **비공개**로 두는 것이 맞습니다. 이번 실습은 브라우저에서 `resizedUrl`을 직접 열어 보는 확인 단계가 있기 때문에 임시로 공개를 택했습니다.

### 5.3.2 IAM 사용자와 Access Key

Spring 서버가 Presigner를 만들 때 쓸 자격 증명을 준비합니다. 콘솔 검색창에서 `IAM`으로 이동한 뒤 "사용자 → 사용자 생성"으로 새 사용자를 만듭니다. 사용자 이름은 실습 식별용이면 충분합니다 (예: `presign-demo-user`).

만들어진 사용자를 클릭해서 **권한 탭 → 권한 추가 → 직접 정책 연결**에서 `AmazonS3FullAccess`를 체크합니다. 실습에서는 모든 권한을 다 주지만, 실서비스에서는 `s3:PutObject`와 `s3:GetObject`만 허용하는 최소 권한 정책을 따로 만들어 쓰는 것이 원칙입니다.

이제 같은 사용자의 **보안 자격 증명** 탭으로 내려가 "액세스 키 만들기 → 로컬 코드"를 선택한 뒤 `.csv`로 내려받습니다. 이 CSV 한 장이 **Access Key**와 **Secret Key**를 담고 있습니다. 다시 볼 수 없는 값이니, 분실 시 재발급이 전제입니다.

> **Access Key가 깃허브에 올라가면**
>
> 과거에 여러 번 발생한 실제 사고 유형입니다. 누군가가 `application.properties`에 키를 그대로 박고 커밋·푸시하면, 크롤러가 몇 분 안에 발견해서 크립토마이닝·무차별 호출로 요금을 수십만 원 단위로 불립니다. 이 챕터에서 `.env`를 쓰는 이유는 단순한 관습이 아니라 **사고 예방 장치**입니다. `.gitignore`에 `.env`가 있는지 꼭 확인하세요.

### 5.3.3 Lambda 함수와 S3 트리거

콘솔에서 `Lambda`로 이동해 "함수 생성"을 클릭합니다.

- 함수 이름: `image-resize-handler`
- 런타임: **Python 3.11**
- 기본 실행 역할: "기본 실행 역할 변경 → AWS 정책 템플릿에서 새 역할 생성"

함수가 만들어지면 **두 가지**를 덧붙여야 합니다. 하나는 이 함수가 S3를 읽고 쓸 수 있도록 **실행 역할에 정책 추가**, 다른 하나는 S3의 `original/` 폴더를 감시하는 **트리거 등록**입니다.

**실행 역할 정책 추가**. `IAM → 역할`로 가서 방금 Lambda가 만든 역할(이름에 함수명이 들어 있습니다)을 찾아 권한 정책에 `AmazonS3FullAccess`를 추가합니다. 실습용이라 그대로 쓰지만, 실서비스에서는 `s3:GetObject`(original/) + `s3:PutObject`(resized/) 두 가지만 허용하는 맞춤 정책이 맞습니다.

**트리거 등록**. Lambda 함수 화면에서 "트리거 추가 → S3 선택"으로 아래처럼 설정합니다.

| 항목 | 값 |
| --- | --- |
| Bucket | 방금 만든 버킷 |
| Event types | `PUT` (또는 `All object create events`) |
| Prefix | `original/` |
| Suffix | (비워 둠) |

`original/`로 시작하는 객체가 만들어질 때만 이 함수가 불립니다. `resized/`에 Lambda 자신이 다시 쓰는 파일로는 절대 재귀 호출이 일어나지 않도록 Prefix가 방지막 역할을 합니다. **이 Prefix를 비우면 무한 루프**가 발생해서 AWS 요금이 치솟습니다. 실습 중 가장 흔한 사고 지점이니 꼭 확인하세요.

트리거가 등록되면 Lambda 함수 화면 왼쪽 "트리거" 항목에 S3 버킷 이름이 보입니다. 코드는 아직 비었지만, 이제 업로드만 일어나면 이 함수가 시도는 할 겁니다.

### 5.3.4 .env 작성

레포 루트에 `.env` 파일을 만들고 아래처럼 네 줄을 채웁니다. 값 자리의 따옴표는 그대로 둬도 됩니다.

실습 2 .env. AWS 자격 증명과 버킷 이름

```
# ----------------------------
# AWS 환경변수
# ----------------------------
CLOUD_AWS_CREDENTIALS_ACCESS_KEY="AKIA..."
CLOUD_AWS_CREDENTIALS_SECRET_KEY="..."
CLOUD_AWS_REGION=ap-northeast-2
CLOUD_AWS_S3_BUCKET="metacoding-base64"
```

`ap-northeast-2`는 서울 리전입니다. 버킷을 서울에 만들었다면 이 값을 그대로 두고, 다른 리전에 만들었다면 콘솔 상단의 리전 코드와 맞춥니다.

## 5.4 Spring에 S3Presigner 얹기

이 절은 네 조각으로 나뉩니다. AWS SDK 의존성 추가 → `S3Presigner` 빈 등록 → 요청·응답 DTO → 발급 서비스/컨트롤러 순서입니다.

### 5.4.1 AWS SDK v2 의존성

`build.gradle`을 열고 `dependencies` 블록에 한 줄을 보탭니다.

실습 3 build.gradle. AWS SDK v2 추가

```
dependencies {
    // ... 기존 의존성은 그대로 둡니다
    implementation 'software.amazon.awssdk:s3:2.25.31'
}
```

이 한 줄이 `S3Client`·`S3Presigner`·`PutObjectRequest` 같은 AWS SDK v2 클래스를 전부 당겨 옵니다. v1(`com.amazonaws:aws-java-sdk-s3`)이 아닌 **v2**를 쓰는 것이 중요합니다. Presigner API의 모양이 v1·v2가 다릅니다.

### 5.4.2 S3Client와 S3Presigner 빈 등록

Spring이 시작될 때 자격 증명을 끼운 두 클라이언트를 한 번만 만들어 두고, 필요한 곳에서 주입받아 쓰도록 합니다.

`src/main/java/com/metacoding/spring_presign_url/_core/config/AwsS3Config.java`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 4 src/main/java/com/metacoding/spring_presign_url/_core/config/AwsS3Config.java. S3 설정

```
@Configuration
public class AwsS3Config {

    // TODO: .env에서 넘어온 자격 증명을 주입받아 두 클라이언트를 빈으로 등록합니다
    // 1. application.properties에 매핑된 환경변수를 필드로 주입
    @Value("${cloud.aws.credentials.access-key}")
    private String accessKey;

    @Value("${cloud.aws.credentials.secret-key}")
    private String secretKey;

    @Value("${cloud.aws.region}")
    private String region;
```

```
    // 2. Access/Secret Key 기반 자격 증명 제공자
    private StaticCredentialsProvider credentialsProvider() {
        AwsBasicCredentials credentials = AwsBasicCredentials.create(accessKey, secret
Key);
        return StaticCredentialsProvider.create(credentials);
    }

    // 3. 일반 S3 작업용 (삭제·조회 등)
    @Bean
    public S3Client s3Client() {
        return S3Client.builder()
                .credentialsProvider(credentialsProvider())
                .region(Region.of(region))
                .build();
    }

    // 4. Presigned URL 서명 전용
    @Bean
    public S3Presigner s3Presigner() {
        return S3Presigner.builder()
                .credentialsProvider(credentialsProvider())
                .region(Region.of(region))
                .build();
    }
}
```

`S3Client`와 `S3Presigner`는 같은 자격 증명·같은 리전을 쓰지만, 역할이 나뉘어 있습니다. `S3Client`는 실제 API 호출(파일 업로드·삭제·조회)을 하는 쪽이고, `S3Presigner`는 오직 **서명된 URL을 만들어서 돌려주는** 쪽입니다. 이번 챕터에서 클라이언트가 S3에 PUT을 쏘는 경로는 `S3Client`를 **거치지 않습니다**. 서버 힙에 파일이 안 올라오는 이유가 여기서도 나옵니다.

### 5.4.3 요청·응답 DTO

Presign 발급 한 건의 입출력을 record로 잡습니다.

`src/main/java/com/metacoding/spring_presign_url/image/ImageRequest.java`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 5 src/main/java/com/metacoding/spring_presign_url/image/ImageRequest.java. 발급 요청

```
public class ImageRequest {

    // TODO: Presign 발급 요청 DTO
```

```
    // 1. fileName — 원본 파일명 ("cat.png"). 확장자 추출의 근거
    // 2. contentType — MIME 타입 ("image/png"). 업로드 시 헤더와 일치해야 함
    public record PresignRequest(
            String fileName,
            String contentType
    ) {}
}
```

`src/main/java/com/metacoding/spring_presign_url/image/ImageResponse.java`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 6 src/main/java/com/metacoding/spring_presign_url/image/ImageResponse.java. 발급 응답

```
public class ImageResponse {

    // TODO: Presign 발급 응답 DTO
    // 1. key — S3 객체 키 (original/{uuid}.ext). 완료 저장 단계에서 다시 쓰임
    // 2. presignedUrl — 서명된 PUT URL (15분 유효)
    public record PresignedUrlResponse(
            String key,
            String presignedUrl
    ) {}
}
```

클라이언트 입장에서 이 응답의 **두 필드**가 모두 중요합니다. `presignedUrl`은 바로 쏠 업로드 주소고, `key`는 업로드가 끝난 뒤 `/complete`를 호출할 때 서버에 돌려줄 식별자입니다. 이 key 하나로 서버는 UUID·경로를 복원합니다.

### 5.4.4 발급 서비스

Presign 발급은 네 단계입니다. UUID 생성 → 확장자 추출 → key 조립 → `S3Presigner`로 서명.

`src/main/java/com/metacoding/spring_presign_url/image/ImageService.java`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 7 src/main/java/com/metacoding/spring_presign_url/image/ImageService.java. Presigned URL 발급

```
@Service
@RequiredArgsConstructor
public class ImageService {
```

```
    @Value("${cloud.aws.s3.bucket}")
    private String bucket;

    private final S3Presigner presigner;

    // TODO: Presigned PUT URL을 만들어 돌려줍니다
    public ImageResponse.PresignedUrlResponse generatePresignedUrl(ImageRequest.Presig
nRequest reqDTO) {
        // 1. 충돌 없는 객체 식별자
        String uuid = UUID.randomUUID().toString();

        // 2. 원본 파일명에서 확장자만 뽑는다 ("cat.png" → "png")
        String ext = reqDTO.fileName()
                .substring(reqDTO.fileName().lastIndexOf('.') + 1);

        // 3. S3에 저장될 경로 조립 (original/ 프리픽스로 Lambda 트리거 발동)
        String key = "original/" + uuid + "." + ext;

        // 4. 서명할 PUT 요청의 모양 정의
        PutObjectRequest objectRequest = PutObjectRequest.builder()
                .bucket(bucket)
                .key(key)
                .contentType(reqDTO.contentType())
                .build();

        // 5. 15분 유효한 Presigned URL 발급
        PresignedPutObjectRequest presignedRequest = presigner.presignPutObject(builde
r →
                builder.signatureDuration(Duration.ofMinutes(15))
                        .putObjectRequest(objectRequest));

        return new ImageResponse.PresignedUrlResponse(
                key,
                presignedRequest.url().toString()
        );
    }
}
```

`presigner.presignPutObject(...)`가 이 챕터의 심장입니다. 이 한 호출이 자격 증명·리전·버킷·key·HTTP 메서드·만료시각을 모두 묶어 SHA-256으로 서명한 URL 한 줄을 만듭니다. URL에는 다음과 같은 쿼리 파라미터가 박힙니다.

- `X-Amz-Algorithm=AWS4-HMAC-SHA256`
- `X-Amz-Date=20251225T203000Z`
- `X-Amz-Expires=900` (900초 = 15분)
- `X-Amz-Signature=a1b2c3...`

- `X-Amz-Credential=AKIA.../20251225/ap-northeast-2/s3/aws4_request`

S3 서버는 이 쿼리 파라미터들을 보고 "이 URL이 진짜 우리 쪽 Access Key로 서명된 것인지, 아직 유효한지, HTTP 메서드가 PUT인지"를 검증한 뒤 업로드를 받습니다. 서명 한 번으로 15분 동안만 유효한 **1회용 입장권**이 만들어지는 셈입니다.

`Duration.ofMinutes(15)`는 실습 편의를 위한 값입니다. 실서비스에서는 업로드 행위가 얼마나 걸릴지를 예상해서 **가능한 짧게** 잡는 것이 맞습니다 (대개 5~10분). 길수록 유출 시 피해창이 넓어집니다.

### 5.4.5 발급 컨트롤러

`src/main/java/com/metacoding/spring_presign_url/image/ImageController.java`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 8 src/main/java/com/metacoding/spring_presign_url/image/ImageController.java. 발급 엔드포인트

```
@RestController
@RequiredArgsConstructor
public class ImageController {

    private final ImageService imageService;

    // TODO: Presigned URL 발급 엔드포인트
    @PostMapping("/presigned")
    public ImageResponse.PresignedUrlResponse presign(@RequestBody ImageRequest.Presig
nRequest reqDTO) {
        return imageService.generatePresignedUrl(reqDTO);
    }
}
```

`/complete`와 `/list`·`/{id}`는 5.7·5.8에서 얹습니다. 이 시점까지는 컨트롤러에 `/presigned` 하나만 있습니다.

### 5.4.6 application.properties 매핑

`src/main/resources/application.properties`에 환경변수 바인딩을 추가합니다.

실습 9 src/main/resources/application.properties. AWS 설정 매핑

```
# ----------------------------
# AWS 환경변수 (.env에서 로드)
# ----------------------------
cloud.aws.credentials.access-key=${CLOUD_AWS_CREDENTIALS_ACCESS_KEY}
cloud.aws.credentials.secret-key=${CLOUD_AWS_CREDENTIALS_SECRET_KEY}
cloud.aws.region=${CLOUD_AWS_REGION}
cloud.aws.s3.bucket=${CLOUD_AWS_S3_BUCKET}
```

`${}` 안의 이름은 `.env` 파일 변수명과 **대소문자까지** 똑같아야 합니다. `AwsS3Config`의 `@Value("${cloud.aws.credentials.access-key}")`가 여기서 한 번 변환된 프로퍼티 이름을 찾아갑니다.

## 5.5 Postman으로 Presigned URL 받고 S3에 직업로드

서버를 띄웁니다.

터미널 실험 5-1 실행. Spring 서버 기동

```
./gradlew bootRun
```

`Started SpringPresignUrlApplication in 3.xxx seconds`가 찍히면 준비 완료입니다.

[CAPTURE NEEDED: ./gradlew bootRun 실행 후 "Started SpringPresignUrlApplication"과 "Tomcat started on port 8080"이 찍힌 터미널 화면]

### 5.5.1 Presigned URL 발급

Postman에서 새 요청을 하나 엽니다.

- 메서드: `POST`
- URL: `http://localhost:8080/presigned`
- Body 탭 → raw → JSON

```
{
```

```
  "fileName": "cat.png",
  "contentType": "image/png"
}
```

[CAPTURE NEEDED: Postman에서 POST http://localhost:8080/presigned에 Body(raw, JSON)로 fileName과 contentType이 들어간 JSON을 붙인 화면]

`Send`를 누르면 200 OK와 함께 아래와 같은 응답이 돌아옵니다.

[CAPTURE NEEDED: Postman 응답 창. 200 OK 상태에 key="original/c5b8f37c-....png", presignedUrl="https://metacoding-base64.s3.ap-northeast-2.amazonaws.com/original/c5b8f37c-....png?X-Amz-Algorithm=..." 가 찍힌 JSON]

`presignedUrl` 값이 한 줄이지만 꽤 깁니다. `?` 뒤부터 이어지는 `X-Amz-Algorithm`·`X-Amz-Date`·`X-Amz-Expires`·`X-Amz-Signature`가 앞 절에서 말했던 서명 쿼리 파라미터들입니다. 이 URL이 서명된 그 15분 동안, 그 key로, PUT 메서드로만 업로드를 받는 전용 주소입니다.

### 5.5.2 S3에 직업로드

새 Postman 요청을 하나 더 엽니다.

- 메서드: `PUT`
- URL: 방금 응답의 `presignedUrl`을 통째로 복사해서 붙여넣기
- Header 탭 → `Content-Type: image/png` 추가 (발급 요청의 `contentType`과 정확히 일치)
- Body 탭 → binary → 실제 PNG 파일 선택

[CAPTURE NEEDED: Postman PUT 요청. URL 창에 긴 presignedUrl 문자열이 들어 있고, Headers 탭에 Content-Type: image/png가 설정되고, Body 탭의 binary가 선택되어 cat.png 파일이 선택된 화면]

`Send`를 누릅니다. 200 OK가 돌아옵니다. 응답 바디는 비어 있습니다. S3는 업로드가 성공하면 본문을 거의 내려 주지 않습니다.

[CAPTURE NEEDED: Postman에서 S3 PUT 업로드 성공 후 200 OK 응답과 빈 바디가 찍힌 화면]

### 5.5.3 S3 콘솔에서 업로드 확인

AWS 콘솔의 버킷 화면으로 가서 `original/` 폴더를 열면, 방금 올린 `c5b8f37c-....png` 파일이 한 건 들어앉아 있는 것이 보입니다.

[CAPTURE NEEDED: AWS S3 콘솔의 metacoding-base64/original 폴더에 c5b8f37c-....png 파일 한 건이 있는 화면]

파일을 클릭해 "객체 URL"을 웹 브라우저에 붙이면, 공개 정책을 열어 둔 덕분에 사진이 그대로 렌더링됩니다.

[CAPTURE NEEDED: 브라우저 주소창에 https://metacoding-base64.s3.ap-northeast-2.amazonaws.com/original/c5b8f37c-....png를 입력해 고양이 원본 사진이 뜬 화면]

여기까지가 챕터 4 대비 가장 큰 변화입니다. **10MB짜리 사진이 Spring 서버의 힙에 한 번도 올라오지 않았습니다.** Spring이 한 일은 URL 한 줄을 만들어 돌려준 것뿐이었습니다.

**그림 5-3. 챕터 4 Base64 vs 챕터 5 Presigned URL**

| BASE64 (CH04) | PRESIGNED (CH05) |
|---|---|
| **서버 힙 점유** 요청당 ≈ 25MB (문자열 + 바이트 배열) | **서버 힙 점유** ≈ 0 (URL 한 줄만) |
| **서버 왕복** 업로드 바이트가 서버를 거쳐 감 | **서버 왕복** 클라이언트 ↔ S3 직접 |
| **10MB 업로드** 3~5초 지연 + OOM 위험 | **10MB 업로드** 네트워크 대역폭이 실질적 상한 |
| **max-request-size** 키워야 함 | **max-request-size** 관계 없음 (S3 기본 5GB) |

서버가 줄어든 책임만큼, 보안(서명·만료)과 파이프라인(이벤트·Lambda)이 그 자리를 대신합니다

동료가 모니터를 힐끗 봤습니다.

**동료** : "서버 콘솔에 로그도 안 찍히네요. 진짜 서버가 안 끼었다는 거네요."

**오픈이** : "끼긴 했어요. 주소 한 줄 만드는 일만요."

그럼 이제 업로드 끝났다는 건 어떻게 알지.

S3에 사진이 떨어진 건 맞지만, 그 사실을 우리 서버가 알 길은 아직 없었습니다. 그리고 사용자에게 보여줄 썸네일도 아직 없었습니다. 두 개를 동시에 풀어 줄 도구가 한 발짝 앞에 있었습니다.

## 5.6 Lambda 리사이즈 함수 배포

5.3.3에서 만든 `image-resize-handler` 함수로 다시 돌아갑니다. 지금까지는 틀만 만들어 뒀고, 코드는 비어 있었습니다. 이제 이 함수 안에 "원본을 읽어서 썸네일을 구워 `resized/`에 올리는" 파이썬 코드를 넣습니다.

Lambda 콘솔에서 함수 화면의 **코드** 탭으로 이동하면 내장 에디터가 열립니다. 거기에 `lambda_function.py`를 선택해서 아래 코드를 통째로 붙여넣고 **Deploy** 버튼을 누릅니다.

실습 10 lambda_function.py. original/ 이벤트를 받아 resized/에 저장

```
import json
import io
import boto3
from PIL import Image

s3 = boto3.client("s3")


def lambda_handler(event, context):
    # 1. 이벤트 원본을 로그로 찍어 둔다 (디버깅 편의)
    print("=== S3 Upload Event ===")
    print(json.dumps(event))

    # 2. 이벤트에서 bucket과 object key를 뽑는다
    bucket = event["Records"][0]["s3"]["bucket"]["name"]
    key = event["Records"][0]["s3"]["object"]["key"]

    # 3. original/ 폴더만 처리한다 (resized/에 쓰는 건 무한 루프 방지)
    if not key.startswith("original/"):
        print("[SKIP] original/ 경로가 아닙니다.")
        return {"status": "ignored"}

    # 4. original/{uuid}.{ext} → uuid만 떼어 낸다
    file_name = key.replace("original/", "")
    uuid = file_name.rsplit(".", 1)[0]

    print(f"[INFO] original key: {key}")
    print(f"[INFO] uuid: {uuid}")

    # 5. 원본을 내려받아 Pillow로 연다
```

```
    original_obj = s3.get_object(Bucket=bucket, Key=key)
    original_bytes = original_obj["Body"].read()

    image = Image.open(io.BytesIO(original_bytes))
    image = image.convert("RGB")          # JPEG 저장 대비 투명도 제거
    image.thumbnail((800, 800))           # 긴 변 기준 800px로 축소

    # 6. 메모리 버퍼에 JPEG로 굽는다
    buffer = io.BytesIO()
    image.save(buffer, format="JPEG", quality=85)
    buffer.seek(0)

    # 7. resized/{uuid}.jpg로 업로드 (이름 규칙이 5.7의 "규칙 신뢰"의 근거)
    resized_key = f"resized/{uuid}.jpg"
    s3.put_object(
        Bucket=bucket,
        Key=resized_key,
        Body=buffer,
        ContentType="image/jpeg",
    )

    print(f"[SUCCESS] 리사이즈 업로드 완료: {resized_key}")

    return {
        "status": "success",
        "originalKey": key,
        "resizedKey": resized_key,
    }
```

파이썬 코드지만 흐름은 Spring과 똑같습니다. 이벤트에서 bucket·key를 꺼내서(2), `original/`로 시작하는 경우만 처리하고(3), UUID를 뽑아 리사이즈 결과 경로를 맞춘(4·7) 다음, 실제 이미지 처리를 하는(5·6) 구조입니다.

`image.thumbnail((800, 800))`은 **긴 변 기준 800px**로 비율을 유지하며 축소합니다. 1920×1080 원본이라면 800×450으로, 4000×3000 원본이라면 800×600으로 나옵니다. `quality=85`는 JPEG 품질 파라미터입니다 (0~100). 85 정도가 용량·품질의 실무 기본값입니다.

### 5.6.1 Pillow Layer 연결

Deploy를 눌러도 함수는 아직 동작하지 않습니다. Lambda Python 3.11 런타임에는 **Pillow가 기본 포함되어 있지 않기** 때문입니다. `from PIL import Image`에서 `ModuleNotFoundError`가 납니다.

해결책은 **Layer**입니다. Layer는 Lambda 함수에 얹는 공용 라이브러리 꾸러미입니다. 내가 직접 빌드할 수도 있지만, 공개된 Klayers 프로젝트가 리전별 ARN을 이미 만들어 두었기 때문에 그 주소를 붙이기만 하면 됩니다.

Lambda 함수 화면의 하단 **계층 (Layers)** 항목에서 "계층 추가 → ARN 지정"을 클릭하고 아래 ARN을 붙입니다.

참고 Klayers Pillow Layer ARN (ap-northeast-2, Python 3.11)

```
arn:aws:lambda:ap-northeast-2:770693421928:layer:Klayers-p311-Pillow:10
```

리전이 다르면 api.klayers.cloud/api/v2/p3.11/layers/latest에서 해당 리전용 ARN을 찾아 교체합니다. 뒤의 `:10`은 Layer 버전입니다. 최신 버전이 바뀌면 숫자가 올라갑니다.

계층이 등록되면 Deploy 상태로 함수가 완전히 준비된 것입니다.

### 5.6.2 다시 업로드해 리사이즈 확인

5.5의 흐름을 다시 한 번 돕니다. `/presigned`로 새 URL 한 장 발급받고, 그 URL로 PUT 업로드를 쏩니다. 이번에는 업로드 성공 후 **3~5초 기다린 뒤** S3 콘솔을 새로고침합니다.

[CAPTURE NEEDED: AWS S3 콘솔. 같은 버킷의 resized/ 폴더 안에 {uuid}.jpg 파일이 새로 생긴 화면. original/의 {uuid}.png와 UUID가 같고 확장자만 jpg로 바뀜]

`resized/{uuid}.jpg`가 생겼다면 파이프라인이 끝까지 한 바퀴 돈 것입니다. 클라이언트가 `original/`에 PUT → S3가 Lambda 트리거 → Lambda가 Pillow로 리사이즈 → `resized/`에 저장이 전부 자동으로 이어졌습니다.

이 결과 JPG를 브라우저에서 열면 800px로 줄어든 축소본이 뜹니다.

[CAPTURE NEEDED: 브라우저에서 https://metacoding-base64.s3.ap-northeast-2.amazonaws.com/resized/c5b8f37c-....jpg를 연 화면. 긴 변이 800px로 축소된 고양이 썸네일]

**Lambda 로그를 직접 읽고 싶을 때**

Lambda 함수 화면 → 모니터링 탭 → "CloudWatch에서 로그 보기"를 클릭하면 `print(json.dumps(event))`로 찍은 S3 이벤트 원본을 그대로 볼 수 있습니다. 업로드한 파일 경로·이벤트 시각·버킷 이름이 전부 여기에 남습니다. 파이프라인이 안 돌 때 제일 먼저 확인할 자리입니다.

## 5.7 Webhook 꿈꾸다 접기, 완료 저장 API

동료가 다시 모니터를 봤습니다.

**동료** : "이제 서버가 'original에 올라온 거 DB에 적어야지' 이걸 어떻게 알아요? Lambda가 서버한테 전화라도 걸어요?"

**오픈이** : "아, 그거 Webhook으로 Lambda가 우리 서버 `/webhook/upload-complete`를 때리면 되죠."

오픈이는 `/webhook/upload-complete` 엔드포인트 설계를 수첩에 적어 내려갔습니다. Lambda 마지막에 `requests.post("http://localhost:8080/webhook/upload-complete", json={...})`를 한 줄 넣는 그림이었습니다.

근데 Lambda에서 localhost가 뭐지.

손이 멈췄습니다. Lambda는 AWS 안에서 돕니다. Lambda의 `localhost:8080`은 Lambda 런타임 자기 자신의 8080일 뿐, 내 노트북의 Spring 서버가 아닙니다. 내 노트북은 공인 IP도 없고, NAT 뒤에 숨어 있어서 바깥에서 도달할 방법이 없었습니다.

개발 단계에서 Webhook이 안 오는구나.

실서비스였다면 Spring 서버가 공인 도메인을 가졌을 테니 Lambda가 그 주소로 `POST`를 쏠 수 있었을 겁니다. 지금은 로컬 개발이라 그 길이 막혔습니다. ngrok 같은 터널링을 잠깐 쓰는 선택지도 있었지만, 실습 단계에서 외부로 구멍을 내는 건 과한 일이었습니다.

팀장이 지나가다 화면을 보고 멈췄습니다.

**팀장** : "서버가 Lambda 결과를 꼭 듣고서야 DB에 적을 수 있어요? Lambda가 뭘 만드는지 미리 정해져 있지 않아요?"

**오픈이** : "`resized/{uuid}.jpg`로 만들어요. 항상 같은 규칙이에요."

**팀장** : "그럼 서버가 Lambda에 안 물어봐도 알지 않아요? 원본 key만 받으면 resized key는 그 규칙대로 맞추면 되니까요."

오픈이는 다시 수첩을 봤습니다. key 규칙이 `original/{uuid}.{ext}` ↔ `resized/{uuid}.jpg`로 **대칭**이었습니다. 이 대칭을 서버가 믿는 한, Lambda의 응답을 듣지 않아도 resized URL을 **계산**할 수 있었습니다.

이건 Webhook이 아니라 규칙 기반이다.

대신 업로드 완료 시점은 **클라이언트**가 알려 줘야 했습니다. 클라이언트는 본인이 PUT 업로드를 성공했다는 사실을 알고 있으니까요. 클라이언트가 "업로드 끝났어, key는 이거야"라고 서버에 `POST /complete`로 알리면, 서버는 그 key로부터 UUID를 뽑고, 규칙에 따라 `originalUrl`·`resizedUrl`을 조합하고, DB에 한 행을 심습니다.

**그림 5-4. Webhook을 버리고, key 규칙을 믿는 쪽으로**




이 설계를 코드로 내립니다.

### 5.7.1 엔티티와 DTO 확장

DB 한 행의 모양이 챕터 4와 달라집니다. 원본과 리사이즈 URL을 둘 다 가져야 합니다.

`src/main/java/com/metacoding/spring_presign_url/image/ImageEntity.java`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 11 src/main/java/com/metacoding/spring_presign_url/image/ImageEntity.java. 두 URL을 가진 한 행

```
@Entity
@Table(name = "image_tb")
public class ImageEntity {

    // TODO: 원본 · 리사이즈 URL을 함께 가진 한 행
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    // 1. uuid — original key에서 추출한 값. 두 URL의 공통 식별자
    private String uuid;

    // 2. fileName — 클라이언트가 올린 원본 이름 ("cat.png")
    private String fileName;

    // 3. originalUrl — S3 정식 URL (https://...amazonaws.com/original/{uuid}.ext)
    private String originalUrl;

    // 4. resizedUrl — Lambda 결과 URL (https://...amazonaws.com/resized/{uuid}.jpg)
    private String resizedUrl;

    // 5. createdAt — 완료 저장 시각
    private LocalDateTime createdAt;
}
```

`src/main/java/com/metacoding/spring_presign_url/image/ImageRequest.java`에 완료 저장용 record를 하나 더 붙입니다.

실습 12 src/main/java/com/metacoding/spring_presign_url/image/ImageRequest.java. 완료 저장 요청

```
public class ImageRequest {

    public record PresignRequest(
            String fileName,
            String contentType
    ) {}

    // TODO: 업로드 완료를 알릴 때 쓰는 DTO
    // 1. fileName — 원본 파일명 (DB 기록용)
    // 2. key — /presigned 응답으로 받았던 original key 그대로 되돌려 줌
    public record CompleteRequest(
            String fileName,
            String key
    ) {}
}
```

`src/main/java/com/metacoding/spring_presign_url/image/ImageResponse.java`에 응답 DTO를 확장합니다.

실습 13 src/main/java/com/metacoding/spring_presign_url/image/ImageResponse.java. 메타 응답

```java
public class ImageResponse {

    public record PresignedUrlResponse(
            String key,
            String presignedUrl
    ) {}

    // TODO: 완료 저장·상세 조회 공용 응답
    public record Detail(
            Long id,
            String uuid,
            String originalUrl,
            String resizedUrl,
            String fileName,
            @JsonFormat(pattern = "yyyy-MM-dd HH:mm") LocalDateTime createdAt
    ) {
        public static Detail fromEntity(ImageEntity e) {
            return new Detail(
                    e.getId(),
                    e.getUuid(),
                    e.getOriginalUrl(),
                    e.getResizedUrl(),
                    e.getFileName(),
                    e.getCreatedAt()
            );
        }
    }

    // TODO: 목록 응답 (가벼운 필드만)
    public record Item(
            Long id,
            String originalUrl,
            String resizedUrl
    ) {
        public static Item fromEntity(ImageEntity e) {
            return new Item(e.getId(), e.getOriginalUrl(), e.getResizedUrl());
        }
    }

    public record Items(List<Item> items) {}
}
```

목록은 상세보다 가벼운 필드(`id`·두 URL)만 내려 줍니다. 프런트에서 썸네일 그리드를 그리는 데는 이 정도로 충분하고, 상세 페이지로 들어갔을 때 `Detail`이 필요합니다.

### 5.7.2 완료 저장 서비스

이 절이 이 챕터에서 가장 짧으면서 가장 중요한 코드입니다. 서버가 Lambda에 아무것도 묻지 않고, **key 규칙만 믿고** resized URL을 조합하는 지점입니다.

`ImageService.java`에 메서드를 하나 더 붙입니다.

실습 14 src/main/java/com/metacoding/spring_presign_url/image/ImageService.java. 완료 저장

```
@Value("${cloud.aws.region}")
private String region;

private final ImageRepository imageRepository;

// TODO: original key만 받아 resized URL을 조합해 DB에 한 행을 심는다
public ImageResponse.Detail checkAndSave(ImageRequest.CompleteRequest reqDTO) {
    String originalKey = reqDTO.key();

    // 1. original/{uuid}.{ext} → uuid만 뽑는다
    String uuid = originalKey.replace("original/", "").split("\\.")[0];

    // 2. Lambda가 저장한 resized 경로를 규칙대로 조립한다
    String resizedKey = "resized/" + uuid + ".jpg";

    // 3. S3 정식 URL 두 개를 만든다
    String originalUrl = "https://" + bucket + ".s3." + region + ".amazonaws.com/" + o
riginalKey;
    String resizedUrl  = "https://" + bucket + ".s3." + region + ".amazonaws.com/" + r
esizedKey;

    // 4. 엔티티로 포장해 DB에 한 행
    ImageEntity entity = ImageEntity.builder()
            .uuid(uuid)
            .fileName(reqDTO.fileName())
            .originalUrl(originalUrl)
            .resizedUrl(resizedUrl)
            .createdAt(LocalDateTime.now())
            .build();

    imageRepository.save(entity);
    return ImageResponse.Detail.fromEntity(entity);
}
```

2번 줄의 `"resized/" + uuid + ".jpg"`가 이 설계의 계약입니다. **Lambda가 이 규칙으로 저장한다**는 것을 서버가 알고, 규칙이 지켜지는 한 서버는 이벤트 응답을 듣지 않고도 resized URL을 계산할 수 있습니다. 규칙이 깨지는 순간(예: Lambda가 확장자를 `.png`로 쓰거나, 경로를 `thumbs/`로 바꾸거나) 이 조립은 존재하지 않는 URL을 만들어 냅니다. 그러니까 **Lambda 코드의 `resized_key = f"resized/{uuid}.jpg"`와 여기 2번 줄은 반드시 같이 움직여야 합니다.**

### 5.7.3 완료 저장 컨트롤러

`ImageController.java`에 엔드포인트를 하나 더 얹습니다.

실습 15 src/main/java/com/metacoding/spring_presign_url/image/ImageController.java. /complete 엔드포인트

```
// TODO: 업로드 완료 알림을 받아 DB에 적는다
@PostMapping("/complete")
public ImageResponse.Detail complete(@RequestBody ImageRequest.CompleteRequest reqDT
O) {
    return imageService.checkAndSave(reqDTO);
}
```

이제 `/presigned`와 `/complete` 두 축이 섰습니다. 하나는 업로드 시작점, 하나는 업로드 종료점입니다. 그 사이에 Lambda 리사이즈가 자동으로 끼어듭니다.

## 5.8 Postman으로 전체 흐름 검증과 목록·상세 조회

### 5.8.1 처음부터 끝까지 한 바퀴

이번에는 Postman에서 세 요청을 연속으로 날립니다.

① `POST /presigned` (5.5.1과 동일). 응답의 `key`와 `presignedUrl`을 복사해 둡니다.

② `PUT {presignedUrl}` (5.5.2와 동일). 200 OK를 받으면 **3~5초 대기**합니다. Lambda가 리사이즈를 끝낼 시간을 줍니다.

③ `POST /complete` (새 요청).

- 메서드: `POST`
- URL: `http://localhost:8080/complete`

- Body 탭 → raw → JSON

```
{
  "fileName": "cat.png",
  "key": "original/c5b8f37c-e767-46b1-97fd-e2d67bd79dff.png"
}
```

`key` 자리에는 ①의 응답에서 받았던 `key` 값을 그대로 붙입니다.

[CAPTURE NEEDED: Postman에서 POST http://localhost:8080/complete에 fileName과 original/{uuid}.png 형태의 key가 들어간 JSON을 Body로 보내는 화면]

`Send`를 누르면 다음 응답이 돌아옵니다.

[CAPTURE NEEDED: Postman 응답. 200 OK에 id, uuid, originalUrl(https://...amazonaws.com/original/{uuid}.png), resizedUrl(https://...amazonaws.com/resized/{uuid}.jpg), fileName, createdAt이 찍힌 JSON]

응답 JSON의 `resizedUrl`을 브라우저에 붙이면 800px로 축소된 JPEG가 뜹니다. `originalUrl`을 붙이면 원본 PNG가 뜹니다. 같은 UUID가 `.png`와 `.jpg` 두 경로를 아우릅니다.

### 5.8.2 H2 콘솔로 DB 확인

```
http://localhost:8080/h2-console
```

에 접속해서 `SELECT * FROM image_tb`를 실행합니다.

[CAPTURE NEEDED: H2 콘솔에서 SELECT * FROM image_tb 결과. 한 행에 id=1, uuid, file_name=cat.png, original_url, resized_url, created_at이 찍힌 화면]

### 5.8.3 목록·상세 조회 API 얹기

`ImageService.java`에 조회 메서드를 두 개 보탭니다.

실습 16 src/main/java/com/metacoding/spring_presign_url/image/ImageService.java. 목록·상세 조회

```
// TODO: 전체 목록 (가벼운 필드만)
public ImageResponse.Items listAll() {
    List<ImageResponse.Item> items = imageRepository.findAll().stream()
            .map(ImageResponse.Item::fromEntity)
            .collect(Collectors.toList());
    return new ImageResponse.Items(items);
}

// TODO: 단건 상세
public ImageResponse.Detail findById(Long id) {
    ImageEntity entity = imageRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("이미지를 찾을 수 없습니다."));
    return ImageResponse.Detail.fromEntity(entity);
}
```

`ImageController.java`에 엔드포인트 두 개를 더합니다.

실습 17 src/main/java/com/metacoding/spring_presign_url/image/ImageController.java. 목록·상세 엔드포인트

```
@GetMapping("/list")
public ImageResponse.Items getAllImages() {
    return imageService.listAll();
}

@GetMapping("/{id}")
public ImageResponse.Detail getImageDetail(@PathVariable Long id) {
    return imageService.findById(id);
}
```

Postman에서 `GET http://localhost:8080/list`와 `GET http://localhost:8080/1`을 차례로 쏴 봅니다.

[CAPTURE NEEDED: Postman에서 GET /list 응답. items 배열 안에 id·originalUrl·resizedUrl만 담긴 가벼운 Item 객체가 들어 있는 JSON]

[CAPTURE NEEDED: Postman에서 GET /1 응답. id=1 한 건의 Detail이 단일 객체로 내려오는 JSON (fileName·createdAt 포함)]

목록과 상세가 다른 DTO를 쓰는 이유는 **전송 비용** 입니다. 썸네일 리스트 화면은 한 번에 수십~수백 건을 받아야 합니다. 이때 `fileName`·`createdAt`·`uuid`까지 매 건에 실어 보내면 페이로드가 필요 이상으로 커집니다. `Item`은 썸네일 리스트 화면에 꼭 필요한 세 필드

(`id` · `originalUrl` · `resizedUrl`)만 담습니다.

**실서비스로 가기 전 점검할 것들**

- **버킷 공개 설정**: 이번 실습은 public 버킷이지만, 실서비스는 비공개 + 조회도 Presigned GET URL로 내려줘야 합니다. 그래야 URL이 유출되어도 권한 만료로 방어됩니다
- **Presigned 만료시간**: `Duration.ofMinutes(15)`는 실습용입니다. 실제로는 5~10분이 적정. 클라이언트의 네트워크 사정과 사용자의 느긋함 사이에서 타협합니다
- `/complete` **검증**: 지금은 클라이언트가 주는 key를 그대로 믿습니다. 실서비스는 `HeadObject`로 실제로 그 key에 객체가 **존재하는지** 확인한 뒤에 DB를 업데이트하는 편이 안전합니다
- **Webhook 복귀**: 실서비스에서 Spring이 공인 도메인을 가지면 Lambda가 직접 서버를 때리는 Webhook 흐름으로 돌아갈 수 있습니다. 그때 이 챕터의 `/complete` API가 그대로 `/webhook/upload-complete`로 이름만 바뀌어 재사용됩니다

### 5.8.4 심화. React 연동과 CORS

이 챕터에서는 다루지 않지만, React 같은 SPA에서 이 파이프라인을 그대로 쓰려면 한 가지가 더 필요합니다. S3가 브라우저의 CORS preflight(`OPTIONS`)에 답해 주지 않으면 PUT 업로드 자체가 브라우저에서 막힙니다. 버킷의 **권한 → CORS 설정**에 React 개발 서버 Origin(`http://localhost:5173`)을 허용하는 규칙을 아래처럼 넣어 두면, Postman으로 됐던 흐름이 브라우저에서도 그대로 돕니다.

```
{
  "AllowedHeaders": ["*"],
  "AllowedMethods": ["PUT", "GET", "HEAD"],
  "AllowedOrigins": ["http://localhost:5173"],
  "ExposeHeaders": []
}
]
```

Spring 쪽 CORS(`CorsConfig`)와 S3 쪽 CORS **둘 다** 맞춰야 합니다. Spring API 호출은 Spring CORS가, S3 직업로드는 S3 CORS가 통제합니다. 하나만 맞춰 두면 절반만 풀립니다. 브라우저 연동은 선택 과제이므로 이 챕터에서는 Postman까지만 검증합니다.

동료가 의자를 반 바퀴 돌렸습니다.

**동료** : "그럼 이제 10MB 사진도 그냥 올라가요. 근데 사용자가 올린 다음에, 리사이즈 끝났는지 프런트는 어떻게 알아요? 3~5초 기다리라고 매번 손으로 기다리게 할 순 없잖아요."

**오픈이** : "지금은 기다렸다가 `/complete` 한 번 쏘는 걸로 때웠어요. 근데 실제로는 그 3~5초 동안 프런트가 뭔가 보여줘야 해요. '처리 중' 상태를요."

**팀장** : "상태를 주기적으로 물어보게 할 수도 있고, 서버가 끝났을 때 밀어 줄 수도 있고, 아예 실시간 채널로 연결해 둘 수도 있죠."

**오픈이** : "그 세 가지가 뭐가 다른지를 비교할 차례네요."

업로드는 해결됐다. 이제는 '끝났다'를 어떻게 알리느냐의 문제.

다음 챕터의 문이 열리고 있었습니다.

## 용어 정리

| 이야기 속 표현 | 진짜 용어 | 정식 정의 |
| --- | --- | --- |
| 호텔 방 키 | Presigned URL | AWS 리소스(S3 객체 등)에 제한된 시간·제한된 작업(HTTP 메서드)으로만 접근할 수 있도록 서명된 임시 URL. AWS SigV4 서명 규격을 따름 |
| 프런트에서 키만 내주기 | Presigned PUT URL 발급 | `S3Presigner.presignPutObject(...)`로 버킷·키·콘텐츠 타입·만료시각을 묶어 서명한 PUT 전용 URL을 생성하는 과정 |
| 짐을 직접 객실에 | 클라이언트 직업로드 (Direct Upload) | 서버를 경유하지 않고 클라이언트가 저장소(S3 등)로 파일을 직접 업로드하는 패턴. 서버는 권한 발급과 메타 기록만 담당 |
| 객실 번호 | Object Key | S3 버킷 안에서 한 객체를 유일하게 식별하는 문자열. 이 실습에서는 `original/{uuid}.{ext}` · `resized/{uuid}.jpg` 형태 |

| | | |
|---|---|---|
| 배정된 한 방, 정해진 기간 | Signature · Expiration | Presigned URL에 포함되는 서명 쿼리(`X-Amz-Signature`)와 만료시각(`X-Amz-Expires`). 위조·기한 초과 요청을 거절하는 근거 |
| 체크인 알림 | S3 Event Notification | 버킷에 객체가 만들어지거나 삭제될 때 Lambda·SQS·SNS로 발행되는 이벤트. 트리거 등록 시 Prefix/Suffix로 범위 제한 가능 |
| 하우스키핑 | AWS Lambda | 이벤트 기반으로 실행되는 서버리스 함수 서비스. 호출이 없으면 과금도 없고, 동시 실행은 AWS가 자동 스케일링 |
| 라이브러리 꾸러미 | Lambda Layer | 함수가 공용으로 참조할 라이브러리·런타임·의존성을 담은 ZIP 아카이브. ARN으로 함수에 부착 |
| 규칙 기반 조립 | Convention over Configuration (약속된 경로 규칙) | `original/{uuid}.ext` ↔ `resized/{uuid}.jpg`처럼 양쪽이 합의한 규칙을 두어, 이벤트 응답 없이도 상대 경로를 계산할 수 있는 설계 원칙 |
| 서버가 직접 전화 | Webhook | 이벤트 발생 측(Lambda)이 수신 측(Spring)의 HTTP 엔드포인트로 `POST`를 쏴 알림을 전달하는 통합 방식. 수신 측이 공인 주소를 가져야 성립 |
| 사용자 전용 출입증 | IAM Access Key / Secret Key | AWS 사용자(또는 역할)가 프로그래밍 방식으로 AWS를 호출할 때 쓰는 자격 증명 쌍. Access Key ID는 공개, Secret Access Key는 비밀 |
| 권한 묶음 | IAM Policy | 어떤 리소스에 어떤 액션을 허용/거부할지를 JSON으로 선언하는 권한 규약. 사용자·역할·그룹에 연결 |

| | | |
|---|---|---|
| 긴 변 기준 축소 | Thumbnail (비율 유지 리사이즈) | 원본의 가로·세로 비율을 유지하면서 긴 변이 지정 픽셀 이하가 되도록 줄이는 이미지 축소 기법. Pillow `Image.thumbnail(size)`이 표준 구현 |

## 이것만은 기억하자

- **서버가 짐을 받지 않는다.** Presigned URL은 "이 키로, 이 메서드로, 이 시간 안에만"이 박힌 1회용 출입증이다. 서버는 이 출입증을 발급만 하고, 실제 업로드는 클라이언트와 S3 사이에서 끝난다
- **키 규칙이 계약이다.** `original/{uuid}.ext` ↔ `resized/{uuid}.jpg` 대칭을 서버가 믿는 순간, 서버는 Lambda의 응답을 듣지 않고도 결과 경로를 계산할 수 있다. 규칙이 깨지면 조용히 "없는 URL"이 DB에 쌓인다
- **Lambda의 로컬 제약을 설계로 돌려라.** 로컬 개발에서 Webhook이 닿지 않는 건 네트워크의 물리 법칙이다. 그 자리를 "클라이언트가 완료를 알린다 + 서버가 규칙으로 조합한다"로 바꾸면 로컬에서도 실서비스에서도 같은 코드가 선다
- **트리거 Prefix를 비우지 마라.** `original/` 없이 트리거를 걸면 Lambda가 자기가 쓴 `resized/`를 다시 받아 무한 루프가 돌고, 분 단위로 요금이 불어난다. 트리거 등록 화면의 Prefix 칸은 항상 채워 넣는다
- **Access Key는 `.env`로, `.env`는 `.gitignore`로.** 깃허브에 키가 노출되면 몇 분 안에 크롤러가 찾아낸다. `application.properties`에 키를 박지 마라
- **다음 챕터에서는** 업로드가 끝났다는 사실을 클라이언트에게 **어떻게 알릴지**를 세 가지 방식으로 비교합니다. 주기적으로 묻게 할지(Polling), 서버가 밀어 줄지(SSE), 아예 양방향 실시간으로 연결해 둘지(WebSocket). 같은 문제를 세 번 풀어 보며 어느 상황에 어느 도구가 맞는지 손으로 체감합니다 (Polling · SSE · WebSocket)